

Project Executable Files

Date	03 July 2026
Team ID	
Project Name	AI Powered Debt Relief & Financial Recovery Platform
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes ,No)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes (SQLAlchemy auto-creates tables)
5	Environment configuration file (.env.example similar) or	Yes
6	Deployed application URL (if hosted)	NA (Local deployment)
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	NA

10	Demo video or walkthrough (if required)	Yes
----	---	-----

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
AI powdered debt relief and financial recovery platform/
├── backend/
│   ├── app/
│   │   ├── routers/
│   │   │   ├── ai_negotiation.py    # AI negotiation letter endpoints
│   │   │   ├── auth.py              # User auth endpoints
│   │   │   ├── dashboard.py         # Dashboard stats endpoint
│   │   │   ├── financial.py         # Financial engine endpoints
│   │   │   └── loans.py             # Loan management CRUD endpoints
│   │   ├── __init__.py              # App package initialization
│   │   ├── ai_service.py            # Google Gemini integration + fallback templates
│   │   ├── auth.py                  # JWT & password hashing logic
│   │   ├── database.py              # SQLite connection & session setup
│   │   ├── financial_engine.py      # Core financial calculation engine
│   │   ├── models.py                # SQLAlchemy ORM models (5 tables)
│   │   └── schemas.py               # Pydantic request/response schemas
│   ├── .env.example                 # Environment variable template
│   ├── finrelief.db                 # SQLite database file
│   ├── main.py                      # FastAPI app entry point
│   └── requirements.txt              # Python dependencies
├── frontend/
│   ├── public/                      # Static assets (favicon, icons)
│   ├── src/
│   │   ├── components/              # Shared UI components
│   │   │   ├── ProtectedLayout.jsx  # Auth-guard wrapper
│   │   │   └── Sidebar.jsx           # Navigation sidebar
│   │   ├── context/                 # React context providers
│   │   │   └── AuthContext.jsx       # Authentication state management
│   │   ├── pages/                   # Route-specific pages
│   │   │   ├── AINegotiation.jsx    # AI letter generator page
│   │   │   ├── Dashboard.jsx        # Home dashboard page
│   │   │   ├── FinancialHealth.jsx  # Health score calculator page
│   │   │   ├── KnowYourRights.jsx   # Consumer rights reference page
│   │   │   ├── Loans.jsx            # Loan management page
│   │   │   ├── Login.jsx            # Sign-in page
│   │   │   ├── Register.jsx         # Sign-up page
│   │   │   └── Settlements.jsx       # Settlement predictor page
│   │   ├── services/                # API interaction layer
│   │   │   └── api.js                # Axios instance & interceptors
│   │   ├── App.jsx                  # Root component with routing
│   │   ├── index.css                 # Global styles
│   │   └── main.jsx                  # React DOM entry point
│   ├── index.html                   # HTML shell
│   ├── package.json                 # Frontend dependencies
│   └── tsconfig.json                 # TypeScript configuration
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Website Link
Login Credentials (Demo)	Create your own account / email: nagaprasanth2007@gmail.com Password : Prasanth@1
Platform / Hosting Provider	Frontend : Vercel Backend : Render
Repository Link	GitHub Link
Demo Video Link	Demo Link

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

1. Install Python 3.10+ from <https://www.python.org/downloads/>
2. Install Node.js & npm from <https://nodejs.org/>
3. Clone the repository: `git clone <repo-url>`
4. **Backend Setup:**
 - Navigate to backend folder: `cd backend`
 - Install dependencies: `pip install -r requirements.txt`
 - Create `.env` file with: `GEMINI_API_KEY=your_api_key_here`
 - Run the backend: `uvicorn main:app --reload`
 - API available at: `http://localhost:8000`
 - API docs at: `http://localhost:8000/docs`
5. **Frontend Setup:**
 - Open new terminal, navigate to frontend: `cd frontend`
 - Install dependencies: `npm install`
 - Start dev server: `npm run dev`
 - Open browser: `http://localhost:5173`
6. Register a new account → Add loans → View dashboard → Generate AI letter

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Gemini API requires valid API key; without it, fallback templates are used	Set GEMINI_API_KEY in .env file; fallback ensures zero downtime
2	No email verification during registration	Planned for future version with OTP/email confirmation
3	Only supports local deployment	Cloud deployment (Render/Vercel/AWS) planned for future